

API



CONTENTS

Version History	3
1. Scope of Documentation	4
1.1 Revolutionizing Your Banking Experience.....	4
1.2 Structure of Collections	4
MSG HOLD	4
MIS POSTING.....	4
1.3 API Flow.....	5
MSG HOLD API.....	5
MIS POSTING API	5
1.5 Technical Pre-requisite required from client's end	6
1.6 Security	6
1.7 Applicable & Reference Document.....	7
2. Ecollection apis	8
2.1 MSG HOLD API.....	8
Introduction	8
Request Parameter	8
Response Parameter.....	9
Sample JSON Packet.....	9
IMPORTANT POINTS:	9
2.2 MIS POSTING API	10
Introduction	10
Request Parameter	10
Response Parameter Details.....	10
Sample JSON Packet.....	10
3. Encryption & Decryption Process	11
3.1 Hybrid Encryption	11
Input parameters	11
ENCRYPTION-DECRYPTION PROCESS	13

VERSION HISTORY

Version	Date	Author	Rationale
1.0	02-12-2025	Rutuja Surve	Added MSG HOLD and MIS POSTING APIs

1. SCOPE OF DOCUMENTATION

1.1 REVOLUTIONIZING YOUR BANKING EXPERIENCE

The banking industry has witnessed a significant leap forward with the advent of API Banking. This groundbreaking technology has not only provided businesses and consumers with easy access to custom banking services but has also ensured enhanced efficiency and complete safety.

In response to the evolving needs of corporates, we proudly present the meticulously curated "ECollection API Suite." This suite of APIs has been thoughtfully designed after studying the corporate journey and requirements in detail. The result is a set of APIs that seamlessly cater to the needs of businesses while automating the reconciliation process, making banking a breeze.

In context to this document, E Collection product facilitates clients to get the funds in Virtual Account Number from payer through inward transactions like NEFT, RTGS, UPI, IMPS & FT in their account.

These APIs can be directly integrated into client business management software/ERP. Response will be reconciled automatically into the ERP through synchronous response.

Ecollection APIs are hosted on the ICICI Bank Gateway. ICICI Bank Gateway will route any request of API to the respective system of the payment mode. ICICI Bank provide separate integration environment for UAT & production environment testing. In the UAT environment, the client will be able to test all possible scenarios & post confirmation; the client will be moved to Production/Live environment. Collaborate with us today to unlock the full potential of API Banking, a revolution that is transforming the banking landscape. Experience the future of banking now!

1.2 STRUCTURE OF COLLECTIONS

MSG HOLD

Client will be able to accept collections from only those remitters/senders, which are validated in their database/ERP. Bank will send payment detail request to client's URL, Client will validate those details with their ERP and provide response to bank whether to accept or reject funds. Here there is dependency on client to provide response to bank.

MIS POSTING

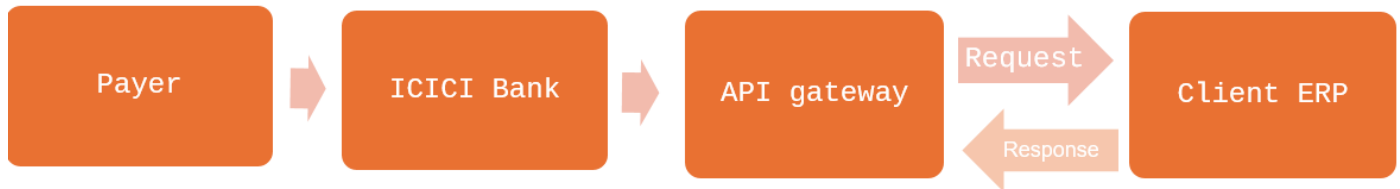
Post successful validation of remitters /sender at clients end through MSG Hold API, Client will get MIS for credited/accepted funds.

Only for credited funds, bank will provide MIS over API to client's URL, Client to consume this API and reconcile/ update the status of transactions in their ERP and provide acknowledgement via API to bank.

1.3 API FLOW

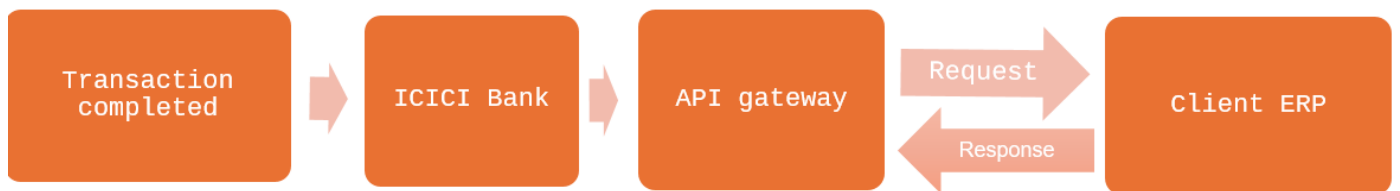
MSG HOLD API

Once the payer initiate transaction using virtual account number. Request will reach to Bank, bank will validate the client code and based on client code bank will send request to client's MSG HOLD URL and expect the response whether to accept or reject the payment received.



MIS POSTING API

Once the transaction is accepted or rejected by the client in the MSG Hold API, then the Bank will send the MIS of the transaction accepted only, and expect response as an acknowledgement.



1.5 TECHNICAL PRE-REQUISITE REQUIRED FROM CLIENT'S END

Pre-requisites	Uses	Remarks
2 call back URLs (https://)	To exchange API request and responses	MSG HOLD API: Validating payer's details from client's server before crediting the amount in client's actual current account MIS POSTING API: To provide credit confirmation to client along with payer's details. Client to consider this API for final confirmation of credit
Public Key certificate (4096 bit)	Public key or CA signed certificate required for encryption at ICICI Bank end	This will be used by ICICI Bank for encryption of API response before sending the same to the client's ERP. Clients can access the API response by decrypting the same using their Private Key. The private key used should be same as the one associated with the given public key certificate. In case of miss-match between public key certificate and private key certificate, the client's will face an error in in encryption & decryption Note: 1) Encryption should happen at client end by using ICICI Bank's public key certificate. The certificate is shared during the integration process. 2) Client can use Open SSL command for creating public key certificate at their end, provided they have access to the server. 3) Certificate provided should be 4096 bit and the format used should be X.509 and the extension used should be (.cer or .crt or .txt). The certificate content should be as follows - Certificate to start with "-----BEGIN CERTIFICATE-----" and end with "-----END CERTIFICATE-----"

1.6 SECURITY

- Bank will provide ICICI bank's public key certificate for encryption of response packet, which is to be done at client end.
- Client's need to whitelist ICICI Bank IPs provided by bank.
- Request will flow from Bank to the client's server via the API gateway.
- All the requests and responses will be encrypted.
- Requests should be in JSON format and should be sent using the POST method. Please find below the sample format
- Requests would be sent using POST method. Please find below the sample format:
`[GatewayURL(Base64Encode(RSA_Encrypt(JSON_Object{Field_Elements(field1,field2,...)})))]`
- The JSON Request Object is mentioned below where complete payload is encrypted using the public key provided by ICICI Bank.
- The encryption logic is to be built by the client as per their convenience.

```
{  
Base64Encode(RSA/ECB/PKCS1Encryption(SesionKey,ClientPubKey.cer))  
}
```

9. Encryption needs to be done using RSA 4096 bits public key provided by ICICI Bank.
10. API request and response to Merchant is secured using advanced and agreed upon RSA encryption algorithm agreed to maintain data confidentiality and integrity.
11. Please refer next section for Encryption & Decryption process.

1.7 APPLICABLE & REFERENCE DOCUMENT

1. ICICI Bank Certificates
2. ICICI Bank public IPs

2. ECOLLECTION APIS

2.1 MSG HOLD API

INTRODUCTION

Client will be able to accept collections from only those remitters/senders, which are validated in their database/ERP. Bank will send payment detail request to client's URL, Client will validate those details with their ERP and provide response to bank whether to accept or reject funds.

REQUEST PARAMETER

Parameter	Datatype (Length)	Mandatory	Description
ClientCode	VARCHAR2	Y	This will be raised in CIB portal
VirtualAccountNumber	VARCHAR2	Y	Created at client's end
Mode	VARCHAR2	Y	Transaction Mode – UPI, IMPS, NEFT,RTGS,FT
USERID	VARCHAR2	Y	User ID under Corporate ID in CIB
UTR	VARCHAR2	Y	Unique Transaction Reference generated for each transaction
SenderRemark	VARCHAR2	Y	If the sender add any remarks
ClientAccountNo	VARCHAR2	Y	Customer Account number
Amount	VARCHAR2	Y	Amount enter by payer
PayerName	VARCHAR2	Y	Payer name as per his/her account details
PayerAccNumber	VARCHAR2	Y	Payer account number
PayerBankIFSC	VARCHAR2	Y	Payer IFSC code
PayerPaymentDate	VARCHAR2	Y	Transaction initiation date
BankInternalTransactionNumber	VARCHAR2	Y	Unique transaction number created at bank end for internal purpose

RESPONSE PARAMETER

Name	Datatype	Mandatory	Description
AcceptOrReject	VARCHAR2	Y	Client needs to validate virtual account number and send Accept or Reject as "Y" or "N"
Message	VARCHAR2	Y	Response message should be "Accept" or "Reject"
Code	VARCHAR2	Y	Response CODE

SAMPLE JSON PACKET

Request Packet	Response Packet
<pre>{ "ClientCode": "ABC", "VirtualAccountNumber": "ABC9834191192", "Mode": "RTGS", "UTR": "ICIC52011986928310711", "SenderRemark": "Testing", "ClientAccountNo": "TEL9834191192", "Amount": "10000.00", "PayerName": "-", "PayerAccNumber": "0008429324", "PayerBankIFSC": "ICIC00000", "PayerPaymentDate": "20251202", "BankInternalTransactionNumber": "REQ20213000001 }</pre>	Accept
	Reject

IMPORTANT POINTS:

DEEMED ACCEPT/DEEMED REJECT SCENARIO:

For MSG Hold API, if client's ERP is unavailable due to some reason, then bank will not do any retry. It will go either for Deemed Accept or Deemed Reject scenario.

- Any one deemed scenario needs to be selected by client before UAT.
- Deemed Accept: By default bank will credit funds to client's account if there is no response from client for MSG Hold API.
- Deemed Reject: Bank will refund back to payer if there is no response from client for MSG Hold API.

Below mentioned are the timelines bank will wait on client's MSG HOLD URL. If bank doesnot get any response in these timelines then it will consider Deemed Accept/Deemed Reject scenario according to the configuration happened at bank end

For UPI: 5 secs, For IMPS: 5secs, For RTGS: 10secs, For NEFT: 10secs, For FT: 15secs

2.2 MIS POSTING API

INTRODUCTION

Post successful validation of remitters /sender at clients end through MSG Hold API, Client will get MIS for credited/accepted funds.

Only for credited funds, bank will provide MIS over API to client's URL, Client to consume this API and reconcile/ update the status of transactions in their ERP and provide acknowledgement via API to bank.

REQUEST PARAMETER

Parameter	Datatype (Length)	Mandatory	Description
ClientCode	VARCHAR2	Y	This will be raised in CIB portal
VirtualAccountNumber	VARCHAR2	Y	Created at client's end
Mode	VARCHAR2	Y	Transaction Mode – UPI, IMPS, NEFT,RTGS,FT
USERID	VARCHAR2	Y	User ID under Corporate ID in CIB
UTR	VARCHAR2	Y	Unique Transaction Reference generated for each transaction
SenderRemark	VARCHAR2	Y	If the sender add any remarks
ClientAccountNo	VARCHAR2	Y	Customer Account number
Amount	VARCHAR2	Y	Amount enter by payer
PayerName	VARCHAR2	Y	Payer name as per his/her account details
PayerAccNumber	VARCHAR2	Y	Payer account number
PayerBankIFSC	VARCHAR2	Y	Payer IFSC code
PayerPaymentDate	VARCHAR2	Y	Transaction initiation date
BankInternalTransactionNumber	VARCHAR2	Y	Unique transaction number created at bank end for internal purpose

RESPONSE PARAMETER DETAILS

Name	Type	Mandatory	Description
RESPONSE	VARCHAR	Y	Success or Duplicate UTR
CODE	VARCHAR	Y	Response code.

SAMPLE JSON PACKET

Request Packet	Response Packet
	Success
<pre>{ "ClientCode": "ABC", "VirtualAccountNumber": "ABC9834191192", "Mode": "RTGS", "UTR": "ICIC52011986928310711", "SenderRemark": "Testing", "ClientAccountNo": "TEL9834191192", "Amount": "10000.00", "PayerName": "-", "PayerAccNumber": "0008429324", "PayerBankIFSC": "ICIC000000", "PayerPaymentDate": "20251202", "BankInternalTransactionNumber": "REQ20213000001 }</pre>	<pre>{ "Response": "Success", "Code": "11" }</pre>
	Duplicate UTR
	<pre>{ "Response": "Duplicate UTR", "Code": "06" }</pre>

3. ENCRYPTION & DECRYPTION PROCESS

3.1 HYBRID ENCRYPTION

For the purpose of maintaining security of APIs, it is very important for the APIs to be sent between the merchant and the bank in an encrypted form. Thus, it is required for the merchant to initiate the API in an encrypted form using their public key. The bank will also be sending the response in the encrypted form and the merchants need to decrypt the same using their private key at their side and take the action accordingly.

INPUT PARAMETERS

Input value & type (maximum character)	Description	Provided by ICICI (Yes/No)	Mandatory value
RequestId [String (64)]	Unique identifier for the request. Not being stored at any level.	--	Optional
Service type [String]	Service Name; Identifying the backend service name	--	Mandatory
EncryptedKey [String. Base64- encoded data (case insensitive)]	One-time use AES key encrypted by the Client's public key. Requirement is for a 128-bit key (with 256-bit key supported as an option).	--	Mandatory

OaepHashingAlgorithm [String (6)]	Describes the algorithm used for Asymmetric Encryption of one time AES key.	Value: NONE, Meaning: RSA/ECB/PKCS1 is used for Asymmetric Encryption Value : SHA1, Meaning : RSA/NONE/OAEPWithSHA1AndMGF1Padding OR RSA/ECB/OAEPWithSHA1AndMGF1Padding	Mandatory
IV [String. Base64-encoded data (case insensitive) (24)]	The initialization vector used when encrypting data using the one-time use AES key.	Exactly 16 bytes actual value to match the block size	Yes, if IV is not part of encrypted data itself. Leave blank otherwise. For the response data encrypted at ICICI end, IV will always be part of encrypted Data itself and it is randomly generated for each request. Can be retrieved by Client by retrieving the first 16 bytes of Base64 decoded encryptedData
EncryptedData [String. Base64-encoded data (case insensitive)]	Contains the encrypted dataPayload object containing the business information. Encrypted by the ephemeral AES key using AES/CBC/PKCS5Padding. Sample unencrypted object: Please refer first section of document for a sample object.	--	Mandatory

ClientInfo [String]	ClientIP or other information	--	Optional
OptionalParam [String]	Reserved for future use	--	Optional

A Sample encrypted response packet sent by ICICI Bank to client (JSON):

```
{
  "requestId": "",
  "service": "",
  "encryptedKey": "pKY7abA+qRizRdYdAmZ8P+O0Pli+4NQSoZSEEEj/kamfyfxqGtleln65SsMjrLSPIXtHJrjrMYo
MphUXoV0zwFMI7v7AMiGgs7y2LntrDxDmOyNk7XTENou00i7sEHmgyZWqS/y0sQQhmk1rWEc9BZaiVBpNPsj
ccAZ9nx4GyRyLvso0pdMrpbpdj8zO80I+5pMAnyf1WcGx5DbZv6fRnl/OddklQeAQASsSG0oFoQcW8pGszg3oD
a6OA0Kbd+7om4CvbcueArU6+MbH3aY3af5isGd0w7/BPTmq8Fei2TbHpSFLRI37nqprRL6+JWk+4eKWd0xVix
xFnNtli4FDh+mnkC6CHBriC4O1hnoSpVken6wl+ltIKNa/FFOE8RucfmBdrHF99eJASuNnU+k8fbcQKPo7gkAtavA
AaN/khaQ/Msa6VEOEWM2IndUpX1PH+dZ1pNFunRWrn813NfodiAtZ1V6KfWT/Rj1DvCsSD/aJgSr3nf1Hz9WU
I9JvAtMeWBwMH+0OaRnrtoJC8Sp76Rfxm/Wb5ddQXavE8Oe4AfvvDK3IZ1hwdQYxa3NFuE9/nTXSOKsmuNrcl
lhnsDNmp6ndjXwr/jlKwH9Fw9i3LYVJL80ng55p9F1VboijVjqhm9FmSIKtmKVgXIL04lytbsm8Z1BHFU3ajvkrUNs
YJrg=",
  "oaepHashingAlgorithm": "NONE",
  "iv": "",
  "encryptedData": "mtlktFRBPPs6SVxh0RWWvsa9bj/W/M56QwvbY/vgYAlolBZuL8L0Qkq8Rxo24U47Tq77U
XDWX+F1KCpWddxGMwnvqCL3k+2b4jjNKsWSVWykXuF3pVRGH4bKZ9gKfRh9C7wQ9x6SuguXUDhjFp+Ds
s7przaNBY0GsRKXR+vUlvR1pwDW0Np+tYvkX7C0n17wg7GGbuWfN5nf92a9xjj512ikE3615pUyDVHBNKeUN
mRHue/UB6IY3nB+/JC/Wro",
  "clientInfo": "",
  "optionalParam": ""
}
```

ENCRYPTION-DECRYPTION PROCESS

For encryption of request at ICICI:

SesionKey = Randomly generated string of length 16 (OR 32).

encryptedKey = Base64Encode (RSA/ECB/PKCS1Encryption (SesionKey,ICICIPubKey.cer))

IV = Initialization Vector- Exactly 16 bytes actual value to match the block size

```
encryptedData = Base64Encode(AES/CBC/PKCS5Padding(Request,SessionKey, IV))
```

For encryption of response at ICICI:

```
encryptedKey= Base64Encode(RSA/ECB/PKCS1Encryption(SesionKey,ClientPubKey.cer))
```

Session key is nothing but randomly generated string of length 16 (OR 32).

```
encryptedData = Base64Encode(AES/CBC/PKCS5Padding(Response,SessionKey))
```

For decryption of response at Client:

```
IV=getFirst16Bytes(Base64Decode(encryptedData))
```

```
SessionKey= Base64Decode(RSA/ECB/PKCS1Decryption(encryptedKey,ClientPrivateKey.p12,))
```

Session key is nothing but randomly generated string of length 16 (OR 32) .

```
Response = Base64Decode (AES/CBC/PKCS5Padding Decryption(encryptedData,SessionKey,IV))
```

OR

Steps for Encryption

Generate 16-digit random number. Say RANDOMNO.

Encrypt RANDOMNO using RSA/ECB/PKCS1Padding and encode using Base64. Say encryptedKey.

Perform AES/CBC/PKCS5Padding encryption on request payload using RANDOMNO as key and ivinitialization vector. Say encryptedData.

Now client may choose to send IV in request from one of below two options.

a.) Send Base64 Encoded IV in "iv" tag. (Recommended Approach)

b.) Send IV as a part of encryptedData itself.

```
bytes[] iv = IV;
```

```
bytes[] cipherText = symmetrically encrypted Bytes (step3)
```

```
bytes[] concatB = iv + cipherText;  
encryptedData = B64Encode(concatB);
```

Perform AES/CBC/PKCS5Padding encryption on DATA using RANDOMNO as key and Base64encoded RANDOMNO as IV. Say ENCR_DATA.

Steps for Decryption

Get the IV- Base64 decode the encryptedData and get first 16 bytes and rest is encrypted response.

```
bytes[] IV= getFirst16Bytes(Base64Decode(encryptedData))
```

Decrypt encryptedKey using algo (RSA/ECB/PKCS1Padding) **and Client's private key.**

Decrypt the response using algo (AES/CBC/PKCS5Padding).

Ignore first 16 bytes of response, as it contains IV.